

OOP Refactoring Lab: Enterprise Payroll Manager

Legacy software maintenance often involves refactoring procedural "Bad Code" scripts into modern, object-oriented architectures. By transitioning from monolithic functions and global state to encapsulated data entities and manager classes, systems become more resilient, testable, and aligned with enterprise-grade engineering standards.

Implement a `PayrollRefactorer` system that modernizes a legacy tuple-based payroll script. The system must utilize an `Employee` data entity to encapsulate individual records and provide a manager class for aggregate calculations, top-earner identification, and structured report generation, all while maintaining strict data encapsulation.

Example 1 Input:

```
ADD E001 50000 5000 ADD E002 60000 2000 REPORT
```

Output:

```
Count: 2 Total: 117000.0 Top: E002
```

Explanation: The refactorer creates `Employee` objects for E001 and E002. It calculates their total payouts (salary + bonus) as 55,000 and 62,000 respectively. The report identifies E002 as the top earner.

Example 2 Input:

```
REPORT (empty dataset)
```

Output:

```
Count: 0 Total: 0.0 Top: No Employees
```

Explanation: The system is designed to be resilient to empty datasets, returning initialized defaults and a "No Employees" status instead of raising errors.

Function Parameter:

- `emp_id` (str): Unique employee identifier.
- `base_salary` (float): Fixed compensation value.
- `bonus` (float): Variable performance compensation.

Returns

- dict: A comprehensive summary containing `count`, `total_payout`, and `top_earner`.

Constraints

- Zero Global State: All data must be stored within class instances.
- Encapsulation: Use an `Employee` class to store specific records.
- Floating point values in the report must be formatted to one decimal place.

Input Format for Custom Testing Input from stdin contains commands: `ADD <id> <base> <bonus>` or `REPORT`.

Sample Case 0 Sample Input

```
ADD E999 100000 25000 REPORT
```

Sample Output

```
Count: 1 Total: 125000.0 Top: E999
```

Explanation A high-compensation record is added, and the report correctly aggregates the single-entry payout.